

10K – Zehntausend

API-Spezifikation | Phase 1.4 – Analyse

Allgemeines

Base-URL (Produktion):	<code>https://kronisoft.net/projekte/10k/api/</code>
Base-URL (Lokal):	<code>http://localhost/projekte/10k/api/</code>
Format:	JSON (Content-Type: application/json)
Authentifizierung:	Token-basiert – jeder Spieler erhält beim Erstellen/Beitreten ein 32-Zeichen-Token (MD5)
Fehlerformat:	<code>{ "ok": false, "error": "Fehlermeldung" }</code>
Erfolgsformat:	<code>{ "ok": true, ... }</code>

Endpunkt-Übersicht

#	Methode	Endpunkt	Funktion	Auth
1	POST	<code>create.php</code>	Spielraum erstellen	Nein
2	POST	<code>join.php</code>	Spielraum beitreten	Nein
3	POST	<code>start.php</code>	Spiel starten (Host)	Token
4	POST	<code>roll.php</code>	Würfeln	Token
5	POST	<code>keep.php</code>	Würfel behalten	Token
6	POST	<code>bank.php</code>	Punkte banken	Token
7	GET	<code>state.php</code>	Spielzustand abrufen (Polling)	Token
8	POST	<code>ai_turn.php</code>	KI-Zug ausführen	Nein

Endpunkt-Details

POST

EP1 – Spielraum erstellen

URL: `api/create.php`

Auth: Nicht erforderlich

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
<code>name</code>	string	ja	Spielername des Hosts (1–30 Zeichen)
<code>ai_count</code>	int	ja	Anzahl KI-Gegner (0–3)

win_score	int	ja	Zielpunktzahl: 5000, 10000 oder 15000
-----------	-----	----	---------------------------------------

Response (JSON):

Feld	Typ	Beschreibung
ok	bool	true bei Erfolg
code	string	6-stelliger Raumcode (z.B. 'AB12CD')
token	string	32-Zeichen Session-Token des Hosts
game_id	int	Datenbank-ID des angelegten Spiels

Beispiel-Response:

```
{ "ok": true, "code": "AB12CD", "token": "a1b2c3d4e5f6...", "game_id": 42 }
```

Fehlerfälle:

HTTP	Fehlerfall
400	Name ungültig oder zu lang
400	ai_count nicht zwischen 0 und 3
400	win_score nicht in [5000, 10000, 15000]

POST

EP2 – Spielraum beitreten

URL: `api/join.php`

Auth: Nicht erforderlich

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
name	string	ja	Spielername des Beitretenden (1–30 Zeichen)
code	string	ja	6-stelliger Raumcode (Groß- oder Kleinschreibung)

Response (JSON):

Feld	Typ	Beschreibung
ok	bool	true bei Erfolg
token	string	32-Zeichen Session-Token des Spielers
game_id	int	Datenbank-ID des Spiels
code	string	Normalisierter Raumcode (Großbuchstaben)

Beispiel-Response:

```
{ "ok": true, "token": "f7e8d9...", "game_id": 42, "code": "AB12CD" }
```

Fehlerfälle:

HTTP	Fehlerfall
------	------------

400	Raumcode nicht 6 Zeichen lang
400	Raum nicht gefunden oder bereits gestartet
400	Raum voll (max. 6 Spieler)

POST

EP3 – Spiel starten (nur Host)

URL: `api/start.php`

Auth: Token erforderlich (muss Host-Token sein)

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
<code>token</code>	string	ja	Session-Token des Hosts

Response (JSON):

Feld	Typ	Beschreibung
<code>ok</code>	bool	true bei Erfolg
<code>started</code>	bool	true

Beispiel-Response:

```
{ "ok": true, "started": true }
```

Fehlerfälle:

HTTP	Fehlerfall
400	Token gehört nicht dem Host (slot != 0)
400	Spiel läuft bereits
400	Weniger als 2 Spieler im Raum

POST

EP4 – Würfeln

URL: `api/roll.php`

Auth: Token erforderlich – nur aktiver Spieler

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
<code>token</code>	string	ja	Session-Token des aktiven Spielers

Response (JSON):

Feld	Typ	Beschreibung
<code>ok</code>	bool	true bei Erfolg
<code>rolled</code>	int[]	Neu gewürfelte Würfelwerte (1–6)
<code>kept</code>	int[]	Bereits behaltene Würfel aus diesem Zug
<code>all_dice</code>	object[]	Alle 5 Würfel mit Feldern v (Wert) und kept (bool)

bust	bool	true wenn kein wertbarer Würfel im Wurf
options	object[]	Verfügbare Wertungsoptionen [{dice, score, label}]
turn_score	int	Bisher gesammelte Punkte in diesem Zug
roll_no	int	Nummer des aktuellen Wurfs in diesem Zug

Beispiel-Response:

```
{ "ok": true, "rolled": [1,3,5,5,2], "kept": [], "bust": false, "options": [{"dice":1,"score":100,"label":"Einzelne 1"}, {"dice":5,5,"score":100,"label":"2x5"}], "turn_score": 0, "roll_no": 1 }
```

Fehlerfälle:

HTTP	Fehlerfall
400	Nicht der aktive Spieler
400	Spiel läuft nicht

POST

EP5 – Würfel behalten

URL: [api/keep.php](#)

Auth: Token erforderlich – nur aktiver Spieler

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
token	string	ja	Session-Token des aktiven Spielers
selected	int[]	ja	Liste der Würfelwerte die behalten werden sollen

Response (JSON):

Feld	Typ	Beschreibung
ok	bool	true bei Erfolg
kept	object[]	Alle bisher behaltenen Würfel [{v: int}]
remaining	int[]	Verbleibende aktive Würfel
turn_score	int	Aktueller Zug-Score nach diesem Keep
score_added	int	Punkte die durch diesen Keep hinzukamen
can_roll	bool	Immer true – Spieler kann weiter würfeln

Beispiel-Response:

```
{ "ok": true, "kept": [{"v":1}, {"v":5}], "remaining": [3,2,6], "turn_score": 150, "score_added": 150, "can_roll": true }
```

Fehlerfälle:

HTTP	Fehlerfall
400	Ausgewählte Würfel nicht verfügbar oder keine Wertung

400	Kein aktiver Wurf vorhanden
-----	-----------------------------

POST

EP6 – Punkte banken

URL: `api/bank.php`

Auth: Token erforderlich – nur aktiver Spieler

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
<code>token</code>	string	ja	Session-Token des aktiven Spielers

Response (JSON):

Feld	Typ	Beschreibung
<code>ok</code>	bool	true bei Erfolg
<code>banked</code>	int	Gebankter Betrag dieser Runde
<code>total</code>	int	Neuer Gesamtscore des Spielers
<code>won</code>	bool	true wenn Spieler die Zielpunktzahl erreicht hat

Beispiel-Response:

```
{ "ok": true, "banked": 450, "total": 3200, "won": false }
```

Fehlerfälle:

HTTP	Fehlerfall
400	Einsteigsbedingung: erster Bank unter 1.000 Punkten
400	Nichts zum Banken (kein Keep zuvor)

GET

EP7 – Spielzustand abrufen

URL: `api/state.php?token={token}`

Auth: Token als URL-Parameter (?token=...)

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
<code>token</code>	string	ja	Session-Token (URL-Parameter, nicht Body)

Response (JSON):

Feld	Typ	Beschreibung
<code>ok</code>	bool	true bei Erfolg
<code>status</code>	string	'lobby', 'running' oder 'finished'
<code>code</code>	string	Raumcode des Spiels
<code>win_score</code>	int	Zielpunktzahl

players	object[]	Alle Spieler: id, slot, name, is_ai, total_score, has_entered
current_slot	int	Slot-Index des aktiven Spielers
my_slot	int	Slot-Index des anfragenden Spielers
my_turn	bool	true wenn der anfragende Spieler dran ist
turn_no	int	Aktuelle Rundenummer
last_roll	object	Letzter Wurf: action, dice_json, kept_json, turn_score
log	object[]	Letzte 10 Aktionen: action, pname, turn_score
winner	string	Name des Gewinners (null wenn Spiel noch läuft)

Beispiel-Response:

```
{ "ok": true, "status": "running", "code": "AB12CD", "win_score": 10000, "players": [{"slot":0,"name":"Emu","is_ai":0,"total_score":1500},...], "current_slot": 0, "my_slot": 0, "my_turn": true, "winner": null }
```

Fehlerfälle:

HTTP	Fehlerfall
401	Ungültiges oder abgelaufenes Token

POST

EP8 – KI-Zug ausführen

URL: `api/ai_turn.php`

Auth: Nicht erforderlich (wird vom Frontend getriggert)

Request-Body (JSON):

Feld	Typ	Pflicht	Beschreibung
game_id	int	ja	Datenbank-ID des Spiels

Response (JSON):

Feld	Typ	Beschreibung
ok	bool	true bei Erfolg
ai_done	bool	true wenn KI-Zug abgeschlossen
busted	bool	true wenn KI einen Bust hatte
banked	int	Gebankter Betrag (0 bei Bust)
won	bool	true wenn KI die Zielpunktzahl erreicht hat (optional)

Beispiel-Response:

```
{ "ok": true, "ai_done": true, "busted": false, "banked": 350 }
```

Fehlerfälle:

HTTP	Fehlerfall
------	------------

400	game_id fehlt oder Spiel nicht aktiv
400	Kein KI-Spieler am aktuellen Slot